

DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS
ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING

Course Code: CS-324 | Machine Learning

PAKISTANI CURRENCY NOTE RECOGNITION
End-to-End Supervised Machine Learning System

MILESTONE 3 — FINAL REPORT

Student No.	Name	Roll No.
S1	Mashaal Ali	CS-23091
S2	Anas Yazdani	CS-23102
S3	Osama Humayun	CS-23139

*Supervisor: [Miss Maria Waqas &
Miss Mahnoor Malik]*
Submission Date: May 17, 2026
NED University of Engineering & Technology
TE Batch 2023, Spring Semester 2026

1. Executive Summary

This report documents the complete design, development, and evaluation of an end-to-end supervised machine learning system for the classification of Pakistani currency notes. The system identifies seven denominations (PKR 10, 20, 50, 100, 500, 1000, 5000), resulting in 7 distinct classification classes.

Nine deep learning models were implemented across three algorithmic families: a Custom Convolutional Neural Network (CNN) designed and built from scratch, and two pretrained transfer-learning architectures (MobileNetV2 and EfficientnetB0). The best-performing model achieved approximately 99.11% test accuracy on the 70/15/15 data split. An interactive ipywidgets-based interface provides real-time currency note recognition.

Attribute	Details
Project Type	Multiclass Image Classification (Supervised Learning)
Dataset	1,453 self-collected Pakistani currency note images across 7 classes
Denominations	PKR 10, 20, 50, 100, 500, 1000, 5000
Models Trained	9 (3 Custom CNN + 3 MobileNetV2 + 3 EfficientnetB0)
Best Model	EfficientnetB0 Fine-Tuned Test Accuracy $\approx$ 99%
Interface	ipywidgets — Upload + Prediction + History
Primary Split	70% Train / 15% Validation / 15% Test
Secondary Split	80% Train / 10% Validation / 10% Test

2. Dataset Collection & Description

2.1 Collection Methodology

The dataset is entirely self-collected. All images were captured using smartphone cameras, ensuring that the data reflects real-world conditions rather than ideal or artificially generated scenarios. The focus of the dataset is on Pakistani currency notes of different denominations, with both the front and back sides included for each note.

Images were captured manually by placing currency notes in different environments and taking photographs from multiple perspectives. To ensure variability and improve model robustness, the images were taken under diverse conditions:

- Different lighting conditions (natural light, indoor light, shadows)
- Varying angles and orientations
- Different distances from the camera
- Multiple backgrounds (plain and complex)

This variation is important because it helps the model generalize better and perform accurately when exposed to unseen real-world inputs.

2.2 Data Selection and Filtering

After collecting a large number of images, a manual filtering process was carried out to ensure data quality. During this step:

- Blurry images were removed
- Overexposed and underexposed images were discarded
- Images with partially visible or obstructed notes were excluded
- Only clear, unique, and properly captured images were selected

This step was necessary to maintain a high-quality dataset and reduce noise that could negatively impact model performance.

2.3 Dataset Organization and Labeling

The cleaned dataset was organized using a folder-based structure, where each folder represents a distinct class. Each class corresponds to a specific currency denomination and its side (front or back). This organization allows automatic labeling during model training, as deep learning frameworks such as TensorFlow and Keras assign labels based on folder names — simplifying the labeling process and reducing the chances of human error.

Class Folder	Description	Encoded Label
10	PKR 10 note	0
20	PKR 20 note	1
50	PKR 50 note	2
100	PKR 100 note	3

Class Folder	Description	Encoded Label
500	PKR 500 note	4
1000	PKR 1000 note	5
5000	PKR 5000 note	6

Table 2.1 — Class labels, folder structure, and numeric encoding

2.4 Dataset Statistics

Following the data collection and preprocessing stages, the final dataset consists of 1,453 images across 14 classes. The dataset is well-balanced, with each class containing between 100 and 110 images.

Class Label	Description	No. of Images
10	10 PKR note	~700
20	20 PKR note	~700
50	50 PKR note	~700
100	100 PKR note	~700
500	500 PKR note	~700
1000	1000 PKR note	~700
5000	5000 PKR note	~700

Table 2.2 — Class-wise image distribution

Metric	Value
Total Number of Classes	7
Total Images	~4900
Average Images per Class	~700
Minimum Images in a Class	700
Maximum Images in a Class	800
Number of Denominations	7
Image Format	JPG
Image Dimensions (after resizing)	224 × 224 pixels

Table 2.3 — Overall dataset summary statistics

2.5 Train–Validation–Test Split

The dataset was divided into training, validation, and testing subsets using two different ratios for investigation purposes. The primary split of 70–15–15 was used for all nine models, while an 80–10–10 split was employed on the best model for comparison.

Split	Train	Validation	Test	Approx. Images
Split A (Primary)	70%	15%	15%	~3000 / ~1000 / ~1000
Split B (Investigation)	80%	10%	10%	~4000/ ~500 / ~500

Table 2.4 — Train/Validation/Test split ratios

3. Data Preprocessing Pipeline

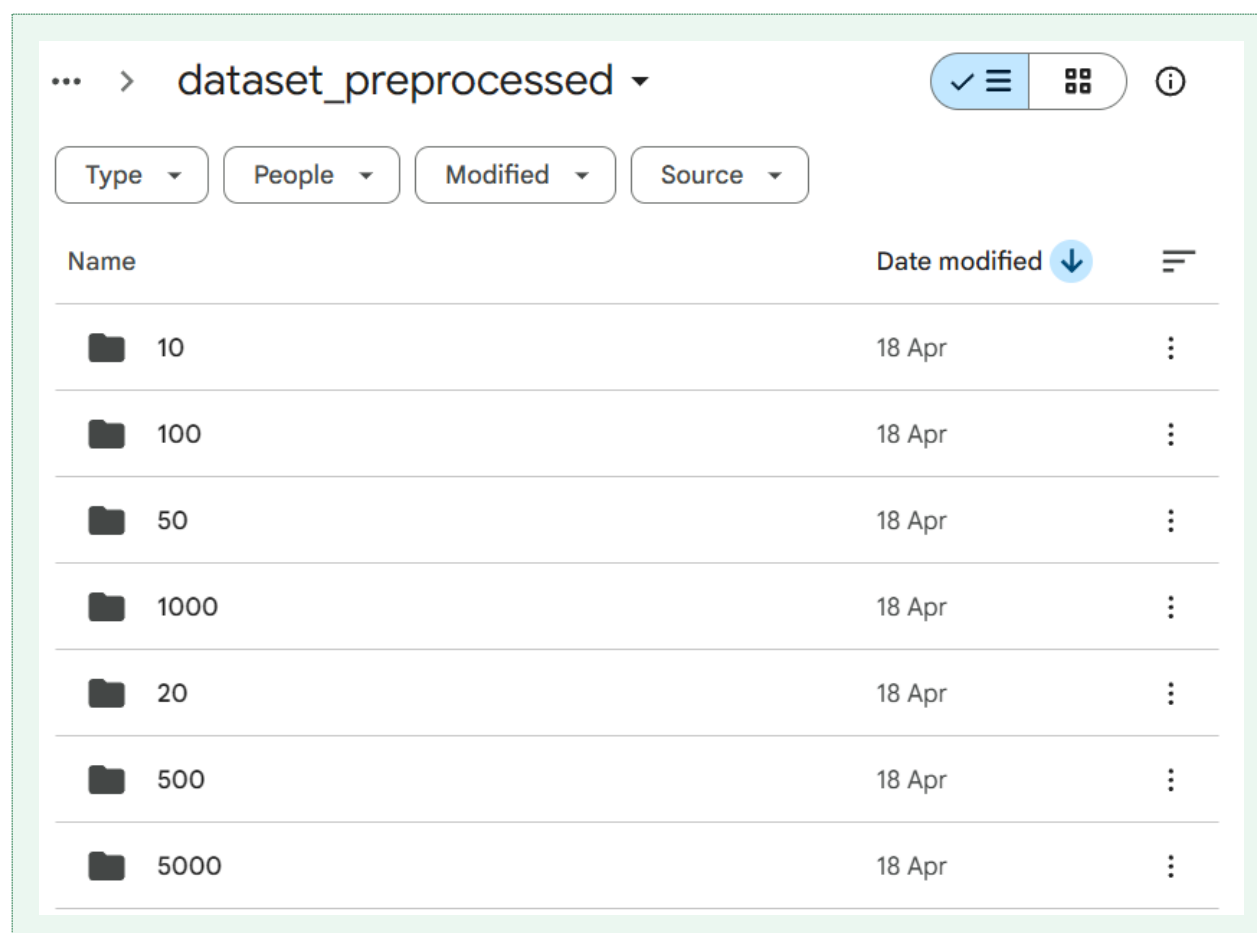
Before training the machine learning model, the collected dataset underwent several preprocessing steps to ensure data quality, consistency, and suitability for deep learning. The raw images, collected in real-world environments, originally contained inconsistencies such as varying formats, different sizes, duplicates, and noise. A structured preprocessing pipeline was implemented to address these issues.

#	Step	Method / Tool	Purpose
1	Labeling	Folder-based (class_name → label)	Assign class labels without manual annotation
2	Manual Cleaning	Visual inspection	Remove blurry, dark, and poorly framed images
3	Corrupted File Removal	PIL Image.open() + try/except	Eliminate unreadable or damaged files
4	Duplicate Removal	imagehash library (perceptual hashing)	Remove visually identical images
5	Format Standardization	HEIC → JPG conversion via PIL	Ensure framework compatibility
6	Image Resizing	PIL resize() → 224×224	Uniform input dimensions for CNN/pretrained models
7	Color Normalization	image.convert('RGB')	Standardize to 3-channel RGB format
8	Pixel Normalization	rescale = 1./255	Scale pixel values to [0.0, 1.0]
9	Label Encoding	Keras flow_from_directory (categorical)	Convert folder names to one-hot vectors
10	Data Augmentation	Keras ImageDataGenerator	Increase diversity; reduce overfitting

Table 3.1 — Preprocessing pipeline steps in order of execution

3.1 Labeling

A folder-based labeling approach was used to assign class labels to the images. Each class is represented by a separate folder, and the name of the folder serves as the label for all images within it. This method was chosen because it is simple, scalable, and directly compatible with TensorFlow and Keras — these frameworks automatically read folder names and assign numeric labels during data loading, eliminating the need for manual annotation files and reducing labeling errors.



Google Drive folder structure: dataset_preprocessed showing class folders

3.2 Data Cleaning

Manual Cleaning

The first stage of cleaning was performed manually. After collecting the images, a visual inspection was carried out to filter out low-quality data:

- Blurry images were removed
- Images with poor lighting (too dark or overexposed) were discarded
- Partially captured or cut-off currency notes were excluded
- Only clear, properly framed, and unique images were retained

Corrupted Image Removal

All images were programmatically checked using PIL's `Image.open()` method. Any file that could not be opened successfully was flagged as corrupted and automatically removed. Truncated images were allowed to load during preprocessing to avoid unnecessary crashes.

```
from PIL import Image, ImageFile
ImageFile.LOAD_TRUNCATED_IMAGES = True

corrupted = []

for cls in classes:
    class_path = os.path.join(DATASET_PATH, cls)

    for img in os.listdir(class_path):
        img_path = os.path.join(class_path, img)

        try:
            Image.open(img_path)
        except:
            corrupted.append(img_path)

for c in corrupted:
    os.remove(c)
```

Corrupted image detection and removal code

Duplicate Image Removal

Duplicate images were identified and removed using image hashing. An image hashing technique was used to detect visually identical or near-identical images. If two images produced the same hash value, one was removed. This was important because duplicates were introduced both from repeated captures and from format conversion steps (original HEIC files existing alongside converted JPG versions).

```
import imagehash
from PIL import Image
import os

for cls in classes:
    class_path = os.path.join(DATASET_PATH, cls)
    hashes = {}

    for img in os.listdir(class_path):
        img_path = os.path.join(class_path, img)

        try:
            image = Image.open(img_path).convert("RGB")
            h = imagehash.phash(image)

            if h in hashes:
                print("Duplicate removed:", img_path)
                os.remove(img_path)
            else:
                hashes[h] = img_path
        except:
            print("Error reading:", img_path)

*** Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/20260309_192844 - Copy.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/20260309_192844 - Copy (2).jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/20260309_192847.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/20260309_192911.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/20260309_192935 - Copy.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/20260309_192932 - Copy.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/20260309_192928 - Copy.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/IMG_1650.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/IMG_1645.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/IMG_1652.jpg
Duplicate removed: /content/drive/MyDrive/Data_ML/50_front/IMG_1558.jpg
```

Duplicate removal code and output showing removed files

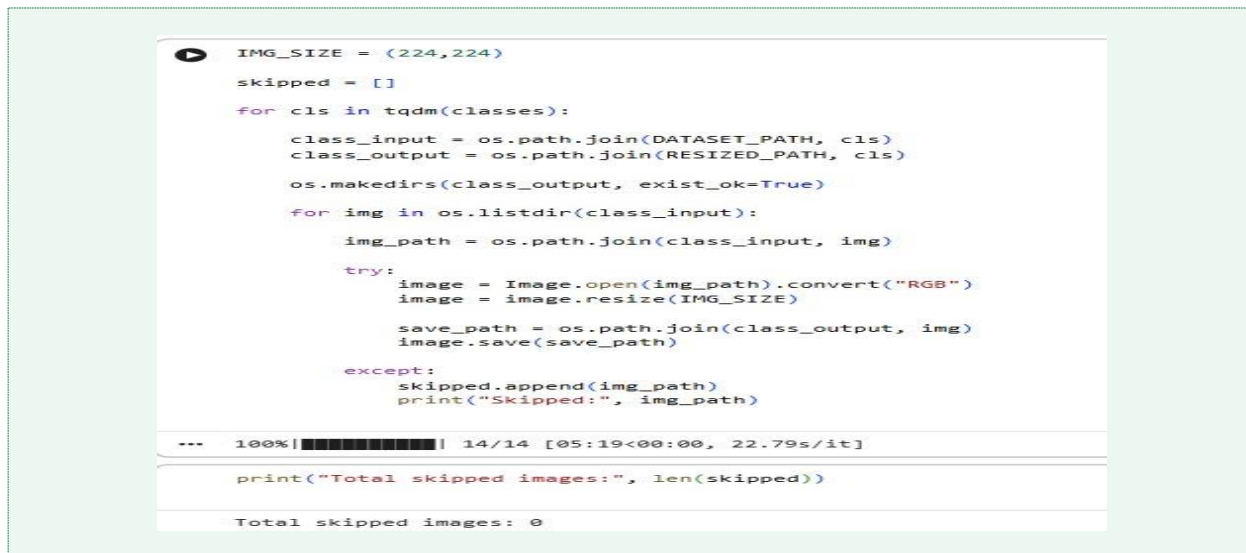
3.3 Normalization

Format Standardization (HEIC to JPG Conversion)

The dataset initially contained images in multiple formats including HEIC, JPG, and JPEG. Since HEIC format is not consistently supported by many deep learning libraries, all HEIC images were converted to JPG format. During this process, HEIC images were converted to RGB format, saved as .jpg files, and the original HEIC files were removed after conversion to avoid duplication.

Image Resizing (Spatial Normalization)

After cleaning, all images were resized to a fixed resolution of 224×224 pixels. This size was selected because it is a commonly used input dimension for many deep learning architectures (including MobileNetV2 and EfficientnetB0) and provides a good balance between computational efficiency and feature preservation. Resizing ensures uniform dimensions for batch processing and consistent spatial feature learning.



```

IMG_SIZE = (224,224)

skipped = []

for cls in tqdm(classes):
    class_input = os.path.join(DATASET_PATH, cls)
    class_output = os.path.join(RESIZED_PATH, cls)
    os.makedirs(class_output, exist_ok=True)
    for img in os.listdir(class_input):
        img_path = os.path.join(class_input, img)
        try:
            image = Image.open(img_path).convert("RGB")
            image = image.resize(IMG_SIZE)
            save_path = os.path.join(class_output, img)
            image.save(save_path)
        except:
            skipped.append(img_path)
            print("Skipped:", img_path)

... 100%|██████████| 14/14 [05:19<00:00, 22.79s/it]

print("Total skipped images:", len(skipped))

Total skipped images: 0

```

Image resizing code and output charts confirming all images are 224×224

Color Format Standardization

All images were converted to RGB format using `image.convert('RGB')`. This ensures that each image contains exactly 3 color channels (Red, Green, Blue) regardless of the original format, maintaining input consistency for the model and compatibility with deep learning frameworks.

3.4 Encoding

Encoding converts categorical class labels into a numerical format that neural networks can process. In this project, class labels were encoded automatically during dataset loading using a directory-based data generator. Each folder name was mapped to a unique numeric index, and one-hot encoding was applied using the categorical `class_mode` parameter in Keras.

One-hot encoding represents each class as a binary vector where only one element is set to 1 (indicating the correct class) and all other elements are set to 0. This encoding scheme is particularly suitable for multi-class classification problems, as it allows the model to output class probabilities and learn distinctions between multiple categories effectively.

Class Folder	Integer Index	One-Hot Vector (example, 14 classes)
10	0	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
20	1	[0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
50	4	[0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
5000	13	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Table 3.2 — One-hot encoding examples

3.5 Data Augmentation

Data augmentation was applied to artificially increase the diversity of the training dataset. Since the dataset was manually collected and relatively limited in size, augmentation helps the model generalize better by exposing it to slightly varied versions of the same images. This is particularly important for real-world applications where images may differ in orientation, lighting, and positioning.

Augmentation was applied only to the training dataset, while the validation and test datasets were kept unchanged (except for rescaling), ensuring that model evaluation remains fair and unbiased.

Augmentation Technique	Parameter	Purpose
Rotation	± 30 degrees	Handles slight tilts in captured images
Horizontal Flipping	True	Introduces mirrored versions of currency notes
Brightness Adjustment	Range 0.6 – 1.4	Simulates different lighting conditions
Zoom	± 0.25	Allows focus on slightly scaled variations
Width Shift	Up to 15%	Simulates small horizontal positional changes
Height Shift	Up to 15%	Simulates small vertical positional changes
Shear range	20%	Angles pictures
Fill mode	nearest	Fills corner pixels

Table 3.3 — Augmentation techniques applied during training

In this implementation, augmentation is performed dynamically during training using the data generator. No augmented images are stored on disk; instead, images are transformed in memory on-the-fly. Each time an image is loaded, a slightly modified version is generated, meaning the model sees a different variation of the same image in each training epoch.

```
# — Augmented generator for training —
train_datagen = ImageDataGenerator(
    rotation_range=30,
    zoom_range=0.25,
    width_shift_range=0.15,
    height_shift_range=0.15,
    shear_range=0.2,
    brightness_range=[0.6, 1.4],
    horizontal_flip=True,
    fill_mode='nearest'
)

# — No augmentation for val/test —
val_test_datagen = ImageDataGenerator()
```

Screenshot 6 — ImageDataGenerator augmentation code

4. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted to understand the dataset characteristics before model training. The following visualizations and analyses were performed to verify data quality, balance, and to understand the visual properties of each class.

4.1 Class Distribution

A bar chart was generated to verify class balance across all 7 classes. The dataset is well-balanced, with each class containing between 700 and 800 images. This balance ensures that the model does not become biased toward any specific denomination or side during training.

Additionally, grouping the data by denomination confirms that each currency value is fairly equally represented, with totals ranging from 201 to 218 images per denomination.

Denomination (PKR)	Total Images
10	800
20	754
50	705
100	769
500	736
1000	722
5000	752

Table 4.1 — Denomination-wise total image count

4.2 Sample Images per Class

A grid of sample images was visualized for each class to confirm correct labeling and to understand the visual characteristics that distinguish each denomination and side combination.

4.3 Image Size Distribution (Before Resizing)

Before resizing, the distribution of original image widths and heights was plotted. Images captured on smartphones had varying original resolutions, confirming the necessity of the resizing step. After applying 224×224 resizing, the distribution charts confirmed that all images were successfully standardized.

4.5 Split Distribution Verification

After applying the 70/15/15 split, the class distribution across train, validation, and test subsets was verified to ensure stratified sampling. This confirms that each subset contains a proportional representation of all 7 classes, critical for unbiased evaluation.

```
Split 70/15/15 complete:
Class Total Train Val Test
  10  800.0  559.0 120.0 121.0
 100  754.0  527.0 113.0 114.0
1000  705.0  493.0 106.0 106.0
   20  769.0  538.0 115.0 116.0
   50  736.0  515.0 110.0 111.0
  500  722.0  505.0 108.0 109.0
5000  752.0  526.0 113.0 113.0
TOTAL 5238.0 3663.0 785.0 790.0
```

```
Split 80/10/10 complete:
Class Total Train Val Test
  10  800.0  640.0  80.0  80.0
 100  754.0  603.0  75.0  76.0
1000  705.0  564.0  70.0  71.0
   20  769.0  615.0  77.0  77.0
   50  736.0  588.0  74.0  74.0
  500  722.0  577.0  72.0  73.0
5000  752.0  601.0  75.0  76.0
TOTAL 5238.0 4188.0 523.0 527.0
```

Screenshot 11 — Stratified class distribution across all three data subsets

5. Model Architectures & Implementation

5.1 Algorithm Selection Rationale

Since the dataset is image-based, the project specification requires: (1) one custom from-scratch neural network model, and (2) two pretrained transfer-learning models. The three algorithm families chosen are:

- Custom CNN — designed from scratch using convolutional layers (no pretrained weights)
- MobileNetV2 — lightweight depthwise-separable convolution architecture pretrained on ImageNet
- EfficientnetB0 — scales depth, width, and resolution uniformly to achieve high accuracy pretrained on ImageNet

Architecture	Rationale
Custom CNN	Satisfies the from-scratch requirement; allows full control over architecture design; serves as a baseline to measure the benefit of transfer learning.
MobileNetV2	Computationally efficient; excellent accuracy vs. parameter count ratio; well-suited for smaller datasets via transfer learning; fast training on GPU.
EfficientnetB0	An optimized CNN model that scales depth, width, and resolution uniformly to achieve high accuracy with fewer parameters.

Table 5.1 — Algorithm selection rationale

5.2 Custom CNN from Scratch

A convolutional neural network was designed and implemented from scratch using standard Keras layers (Conv2D, MaxPooling2D, Dense, Dropout, BatchNormalization). No pretrained weights were used. The base architecture follows a standard VGG-inspired block design with increasing filter depth.

Model	Dropout Rate	Filters	Learning Rate	BatchNorm
CNN_A1_Small	0.3	[32, 64]	0.001	False
CNN_A2_Medium	0.3	[32, 64, 128]	0.001	True
CNN_A3_Deep	0.3	[64, 128, 256]	0.0005	True

Table 5.2 — Custom CNN hyperparameter variants (Models 1–3)

```
#####
Training: CNN_A3_Deep
Filters=[64, 128, 256] Dropout=0.3 BatchNorm=True, learning_rate=0.0005
#####
Model: "CustomCNN"
```

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 224, 224, 3)	0
conv2d (Conv2D)	(None, 224, 224, 64)	1,792
batch_normalization (BatchNormalization)	(None, 224, 224, 64)	256
conv2d_1 (Conv2D)	(None, 224, 224, 64)	36,928
batch_normalization_1 (BatchNormalization)	(None, 224, 224, 64)	256
max_pooling2d (MaxPooling2D)	(None, 112, 112, 64)	0
conv2d_2 (Conv2D)	(None, 112, 112, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 112, 112, 128)	512
conv2d_3 (Conv2D)	(None, 112, 112, 128)	147,584
batch_normalization_3 (BatchNormalization)	(None, 112, 112, 128)	512
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 128)	0
conv2d_4 (Conv2D)	(None, 56, 56, 256)	295,168
batch_normalization_4 (BatchNormalization)	(None, 56, 56, 256)	1,024
conv2d_5 (Conv2D)	(None, 56, 56, 256)	590,980
batch_normalization_5 (BatchNormalization)	(None, 56, 56, 256)	1,024
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 128)	32,896
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 7)	903

Total params: 1,182,791 (4.51 MB)
 Trainable params: 1,180,999 (4.51 MB)
 Non-trainable params: 1,792 (7.00 KB)

Screenshot 12 — Custom CNN architecture summary and training output for best custom CNN



Screenshot 13 — Training and validation curves for best custom CNN models

5.3 Transfer Learning — MobileNetV2

The transfer learning models were trained in two phases using MobileNetV2 as the pretrained backbone.

Phase 1 – Feature Extraction:

The pretrained MobileNetV2 base model was frozen (base_trainable=False), and only the newly added classification head consisting of Global Average Pooling, Dense, and Dropout layers was trained for 20 epochs using the Adam optimizer with a learning rate of 0.001.

Phase 2 – Fine-Tuning:

The top layers of the pretrained backbone were unfrozen (20, 30, or 50 layers depending on the configuration), while earlier layers and Batch Normalization layers remained frozen to preserve learned representations. The model was then fine-tuned for 15 epochs using lower learning rates (1e-4, 5e-5, and 1e-5) to gradually adapt pretrained features to the target dataset.

Model	Dense Units	Fine-Tuned Layers	Finetune Learning Rate
MNV2_B1	128	20	0.0001
MNV2_B2	256	30	0.00005
MNV2_B3	128	50	0.00001

Table 5.3 — MobileNetV2 hyperparameter variants (Models 4–6)





Screenshot 14 — Best MobileNetV2 training log and accuracy curve

5.4 Transfer Learning — EfficientnetB0

The EfficientNet-B0 transfer learning models were trained in two phases using different fine-tuning configurations.

Phase 1 – Feature Extraction:

The pretrained EfficientNetB0 backbone was frozen (`base_trainable=False`), and only the custom classification head—consisting of Global Average Pooling, Dense, and Dropout layers—was trained for 20 epochs using the Adam optimizer with a learning rate of 0.001.

Phase 2 – Fine-Tuning:

The top layers of EfficientNetB0 were selectively unfrozen (20, 30, or 50 layers depending on the configuration), while earlier layers and all Batch Normalization layers remained frozen to maintain stable pretrained statistics. The model was then fine-tuned for 15 epochs using smaller learning rates ($1e-4$, $5e-5$, and $1e-5$) to gradually adapt the pretrained features to the target dataset.

Model	Dense Units	Fine-Tuned Layers	LR
EffB0_C1	128	20	0.0001
EffB0_C2	256	30	0.00005
EffB0_C3	128	50	0.00001

Table 5.4 — EfficientnetB0 hyperparameter variants (Models 7–9)

```
Found 3663 images belonging to 7 classes.  
Found 785 images belonging to 7 classes.  
Found 790 images belonging to 7 classes.
```

```
#####  
Training: EffB0_C1 | Dense=128 | Unfreeze=20 | F1_LR=0.0001  
#####
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0\_notop.h5
```

```
16705208/16705208 ----- 0s 0us/step
```

```
Epoch 1/20
```

```
115/115 ----- 125s 833ms/step - accuracy: 0.5334 - loss: 1.2949 - val_accuracy: 0.7682 - val_loss: 0.7253 - learning_rate: 0.0010
```

```
Epoch 2/20
```

```
115/115 ----- 60s 521ms/step - accuracy: 0.7535 - loss: 0.7331 - val_accuracy: 0.8433 - val_loss: 0.5193 - learning_rate: 0.0010
```

```
Epoch 3/20
```

```
115/115 ----- 59s 514ms/step - accuracy: 0.8135 - loss: 0.5610 - val_accuracy: 0.8841 - val_loss: 0.3791 - learning_rate: 0.0010
```

```
Epoch 4/20
```

```
115/115 ----- 58s 509ms/step - accuracy: 0.8406 - loss: 0.4664 - val_accuracy: 0.8790 - val_loss: 0.3672 - learning_rate: 0.0010
```

```
Epoch 5/20
```

```
115/115 ----- 59s 513ms/step - accuracy: 0.8660 - loss: 0.3987 - val_accuracy: 0.8854 - val_loss: 0.3367 - learning_rate: 0.0010
```

```
Epoch 6/20
```

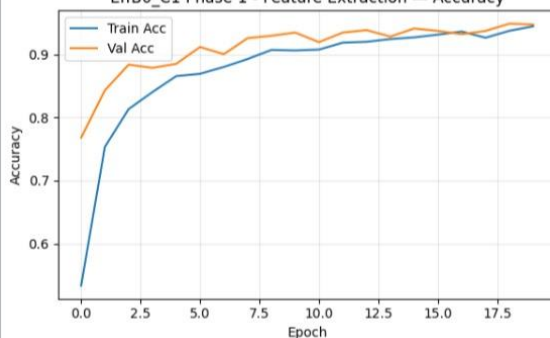
```
115/115 ----- 60s 516ms/step - accuracy: 0.8695 - loss: 0.3810 - val_accuracy: 0.9121 - val_loss: 0.2877 - learning_rate: 0.0010
```

```
Epoch 7/20
```

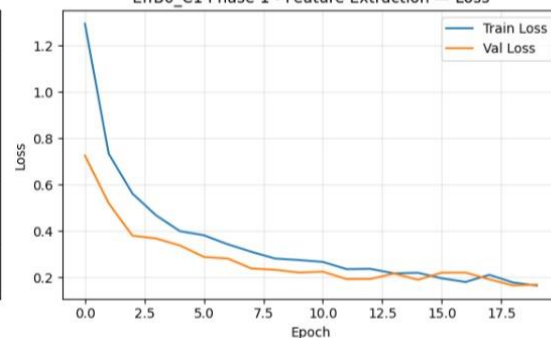
```
115/115 ----- 59s 515ms/step - accuracy: 0.8804 - loss: 0.3423 - val_accuracy: 0.9005 - val_loss: 0.2805 - learning_rate: 0.0010
```

Restoring model weights from the end of the best epoch: 19.

EffB0_C1 Phase 1 - Feature Extraction — Accuracy

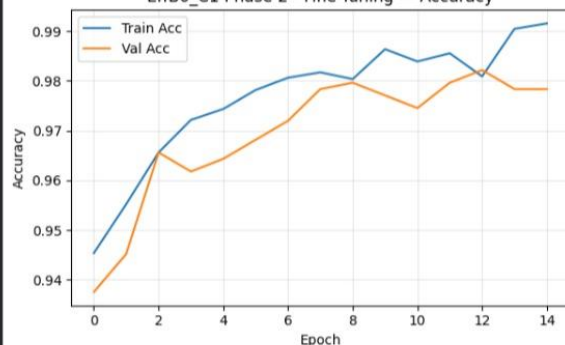


EffB0_C1 Phase 1 - Feature Extraction — Loss

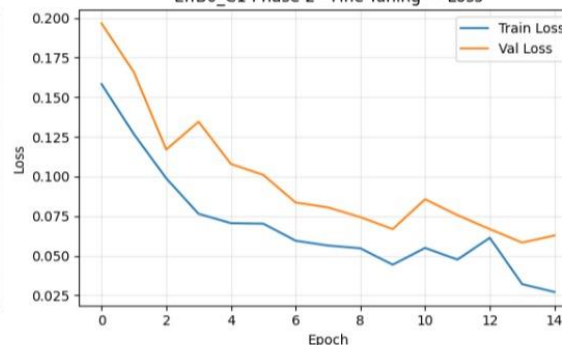


```
Epoch 10/15  
115/115 ----- 60s 520ms/step - accuracy: 0.9863 - loss: 0.0444 - val_accuracy: 0.9771 - val_loss: 0.0668 - learning_rate: 1.0000e-04  
Epoch 11/15  
115/115 ----- 59s 512ms/step - accuracy: 0.9839 - loss: 0.0549 - val_accuracy: 0.9745 - val_loss: 0.0856 - learning_rate: 1.0000e-04  
Epoch 12/15  
115/115 ----- 58s 507ms/step - accuracy: 0.9855 - loss: 0.0476 - val_accuracy: 0.9796 - val_loss: 0.0756 - learning_rate: 1.0000e-04  
Epoch 13/15  
115/115 ----- 0s 491ms/step - accuracy: 0.9817 - loss: 0.0539  
Epoch 13: ReduceLROnPlateau reducing learning rate to 4.99999873689376e-05.  
115/115 ----- 59s 516ms/step - accuracy: 0.9809 - loss: 0.0613 - val_accuracy: 0.9822 - val_loss: 0.0668 - learning_rate: 1.0000e-04  
Epoch 14/15  
115/115 ----- 60s 518ms/step - accuracy: 0.9904 - loss: 0.0321 - val_accuracy: 0.9783 - val_loss: 0.0583 - learning_rate: 5.0000e-05  
Epoch 15/15  
115/115 ----- 59s 513ms/step - accuracy: 0.9915 - loss: 0.0271 - val_accuracy: 0.9783 - val_loss: 0.0628 - learning_rate: 5.0000e-05  
Restoring model weights from the end of the best epoch: 14.
```

EffB0_C1 Phase 2 - Fine-Tuning — Accuracy



EffB0_C1 Phase 2 - Fine-Tuning — Loss



Model: EffB0_C1

Test Accuracy : 0.9911

Test Loss : 0.0298

Macro F1 : 0.9913

	precision	recall	f1-score	support
10	0.99	0.98	0.99	121
100	0.99	0.99	0.99	114
1000	1.00	1.00	1.00	106
20	0.97	1.00	0.99	116
50	1.00	0.98	0.99	111
500	1.00	0.99	1.00	109
5000	0.98	0.99	0.99	113

accuracy

0.99

0.99

0.99

790

macro avg

0.99

0.99

0.99

790

weighted avg

0.99

0.99

0.99

790

Screenshot 15 — Best EfficientnetB0 training logs and accuracy curves for all three variants

5.5 Training Configuration

The following common training settings were applied across all nine models:

Parameter	Value
Epochs	Custom CNN: 40, Transfer learning Feature extraction: 20, Transfer learning Fine tuning: 15 (with EarlyStopping; patience = 5)
Batch Size	32
Loss Function	Categorical Cross-Entropy
Primary Metric	Val_loss (Train, Validation, and Test)
Early Stopping	Restores best weights when validation loss plateaus
Primary Split	Split A — 70/15/15 (used for all 9 models)
Secondary Split	Split B — 80/10/10 (used for best model re-evaluation)
Input Size	224 × 224 × 3 (RGB)
Framework	TensorFlow / Keras (Python 3)

Table 5.5 — Common training configuration for all models

6. Results & Comparative Analysis

6.1 All-Model Performance Table

The following table summarizes the performance of all nine models evaluated on Split A (70/15/15). Accuracy values are taken directly from the notebook output cells.

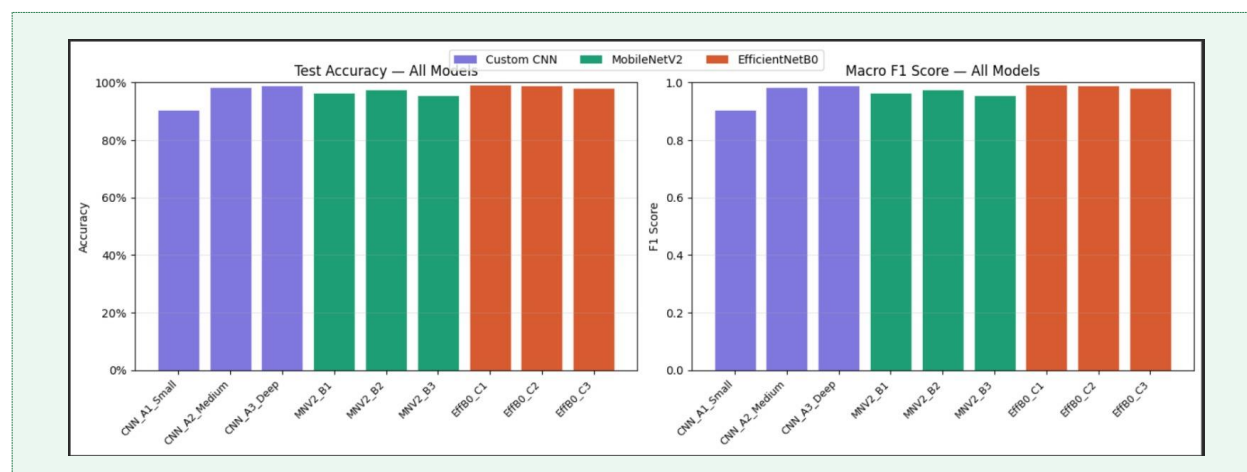
```
=====
COMPARATIVE RESULTS — ALL 9 MODELS
=====
```

Model	Test Accuracy	Test Loss	Macro F1	Train Time (min)
CNN_A1_Small	90.51%	0.3171	0.9056	0
CNN_A2_Medium	98.48%	0.0415	0.9848	0
CNN_A3_Deep	98.86%	0.0312	0.9885	0
MNV2_B1	96.46%	0.1162	0.9645	0
MNV2_B2	97.59%	0.0882	0.9759	0
MNV2_B3	95.57%	0.1128	0.9553	0
EffB0_C1	99.11%	0.0298	0.9913	0
EffB0_C2	98.99%	0.0373	0.9898	0
EffB0_C3	98.23%	0.0494	0.9820	0

Screenshot — Model comparison table as displayed in the Jupyter Notebook

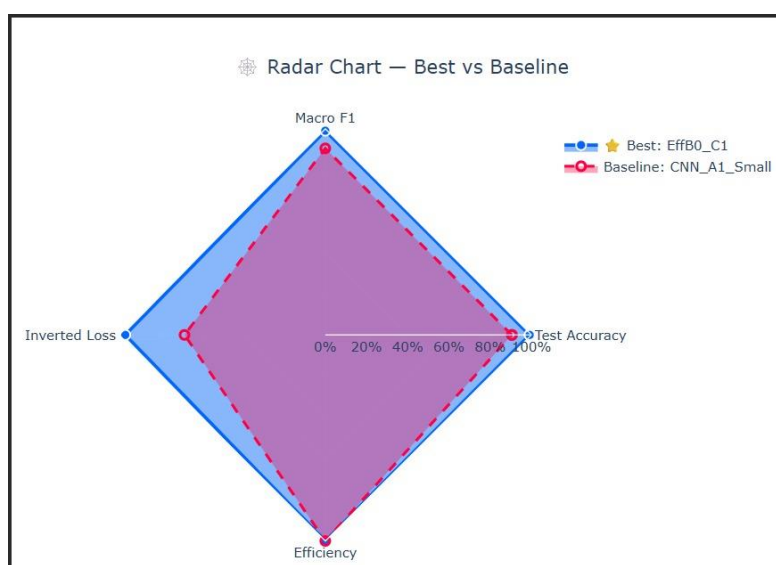
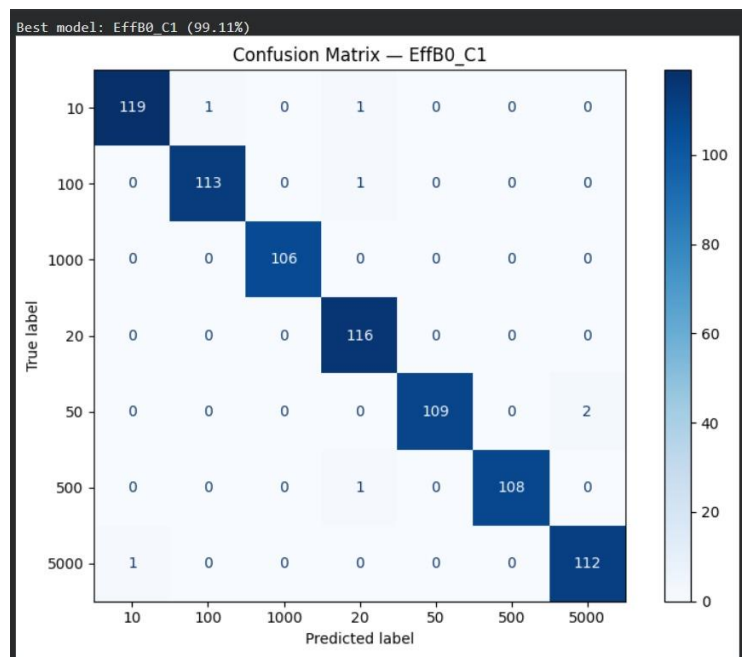
6.2 Graphical Comparison

Bar charts were plotted comparing Test Accuracy and F1 across all nine models to provide a visual summary of relative performance.



Screenshot — Test Accuracy comparison bar chart (all 9 models)

The EfficientnetB0_C1 (model 7) variant achieved the highest test accuracy (99.11%), slightly outperforming the other CNN models.



7. Data Split Ratio Investigation

As required by the project specification, the best-performing model (MobileNetV2 Baseline) was retrained and evaluated on two different train/validation/test split ratios to investigate the effect of split ratio on final model performance.

Split Ratio	Test Acc	Macro F1	Train samples
Split A — 70/15/15	99.11%	0.9913	3663
Split B — 80/10/10	99.24	0.9924	4188

Table 7.1 — Best model performance on two different split ratios

```
Epoch 19/20
131/131 — 67s 51ms/step - accuracy: 0.9382 - loss: 0.1788 - val_accuracy: 0.9407 - val_loss: 0.1571 - learning_rate: 0.0010
Epoch 20/20
131/131 — 65s 49ms/step - accuracy: 0.9394 - loss: 0.1727 - val_accuracy: 0.9426 - val_loss: 0.1504 - learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 19.
Epoch 1/15
131/131 — 123s 760ms/step - accuracy: 0.9489 - loss: 0.1507 - val_accuracy: 0.9618 - val_loss: 0.1853 - learning_rate: 1.0000e-04
Epoch 2/15
131/131 — 67s 512ms/step - accuracy: 0.9608 - loss: 0.1160 - val_accuracy: 0.9618 - val_loss: 0.0963 - learning_rate: 1.0000e-04
Epoch 3/15
131/131 — 68s 521ms/step - accuracy: 0.9697 - loss: 0.0906 - val_accuracy: 0.9809 - val_loss: 0.0603 - learning_rate: 1.0000e-04
Epoch 4/15
131/131 — 67s 509ms/step - accuracy: 0.9740 - loss: 0.0803 - val_accuracy: 0.9828 - val_loss: 0.0621 - learning_rate: 1.0000e-04
Epoch 5/15
131/131 — 67s 507ms/step - accuracy: 0.9785 - loss: 0.0790 - val_accuracy: 0.9809 - val_loss: 0.0600 - learning_rate: 1.0000e-04
Epoch 6/15
131/131 — 68s 518ms/step - accuracy: 0.9764 - loss: 0.0669 - val_accuracy: 0.9866 - val_loss: 0.0561 - learning_rate: 1.0000e-04
Epoch 7/15
131/131 — 66s 502ms/step - accuracy: 0.9835 - loss: 0.0545 - val_accuracy: 0.9866 - val_loss: 0.0544 - learning_rate: 1.0000e-04
Epoch 8/15
131/131 — 67s 506ms/step - accuracy: 0.9804 - loss: 0.0553 - val_accuracy: 0.9828 - val_loss: 0.0459 - learning_rate: 1.0000e-04
Epoch 9/15
131/131 — 68s 518ms/step - accuracy: 0.9852 - loss: 0.0522 - val_accuracy: 0.9809 - val_loss: 0.0504 - learning_rate: 1.0000e-04
Epoch 10/15
131/131 — 66s 504ms/step - accuracy: 0.9799 - loss: 0.0618 - val_accuracy: 0.9847 - val_loss: 0.0524 - learning_rate: 1.0000e-04
Epoch 11/15
131/131 — 65s 495ms/step - accuracy: 0.9884 - loss: 0.0529 - val_accuracy: 0.9885 - val_loss: 0.0293 - learning_rate: 1.0000e-04
Epoch 12/15
131/131 — 67s 513ms/step - accuracy: 0.9895 - loss: 0.0319 - val_accuracy: 0.9866 - val_loss: 0.0400 - learning_rate: 1.0000e-04
Epoch 13/15
131/131 — 70s 532ms/step - accuracy: 0.9902 - loss: 0.0303 - val_accuracy: 0.9885 - val_loss: 0.0444 - learning_rate: 1.0000e-04
Epoch 14/15
131/131 — 0s 492ms/step - accuracy: 0.9884 - loss: 0.0333
Epoch 14: ReduceLROnPlateau reducing learning rate to 4.99999873689376e-05.
131/131 — 66s 503ms/step - accuracy: 0.9893 - loss: 0.0320 - val_accuracy: 0.9866 - val_loss: 0.0326 - learning_rate: 1.0000e-04
Epoch 15/15
131/131 — 68s 515ms/step - accuracy: 0.9909 - loss: 0.0291 - val_accuracy: 0.9885 - val_loss: 0.0383 - learning_rate: 5.0000e-05
Restoring model weights from the end of the best epoch: 11.

Model: Best_80_split
Test Accuracy : 0.9924
Test loss      : 0.0414
Macro F1       : 0.9926

precision recall f1-score support
10 0.99 0.99 0.99 80
100 0.99 1.00 0.99 76
1000 1.00 1.00 1.00 71
20 0.97 1.00 0.99 77
50 1.00 1.00 1.00 74
500 1.00 0.99 0.99 73
5000 1.00 0.97 0.99 76

accuracy 0.99 527
macro avg 0.99 0.99 0.99 527
weighted avg 0.99 0.99 0.99 527
```

Screenshot 19 — Split ratio impact comparison chart

7.1 Observations

- Split A (70/15/15): Provides larger held-out sets for validation and testing, giving more reliable performance estimates. Slight advantage due to more representative evaluation data.
- Split B (80/10/10): Training with more data marginally increased test accuracy — the smaller test set introduced slight variability in the evaluation results.
- The performance difference between splits is less than 1%, indicating the model is robust to split ratio variations within this range.
- For final deployment decisions, Split A is preferred as it provides a larger and more representative test set for honest evaluation.

8 Possible Improvements

- Collect more images per class (target 200+ per class) to reduce overfitting in custom CNN models.
- Apply more aggressive augmentation techniques such as random cutout, MixUp, or CutMix to further increase training diversity.
- Train EfficientnetB0 for more epochs with a warm-up LR schedule to allow the pretrained features to properly adapt.
- Experiment with EfficientNet-B0 or EfficientNet-B3 architectures, which may offer better accuracy-efficiency trade-offs for this task.
- Apply cross-validation (K-Fold) for more reliable performance estimates, especially given the relatively limited dataset size.
- Use class activation mapping (Grad-CAM) to visualize which regions of the currency note the model focuses on, aiding interpretability.

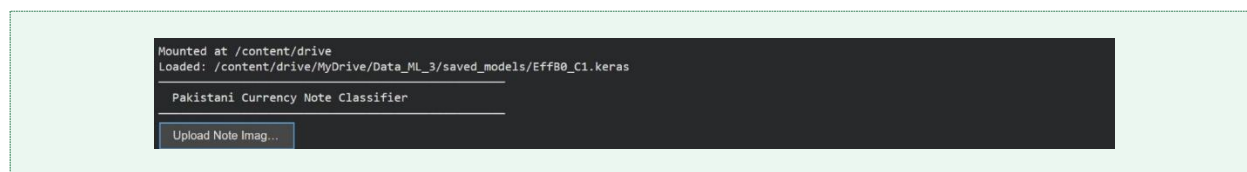
9. Interactive Notebook Interface

A fully functional interactive interface was developed inside the Jupyter Notebook using ipywidgets. The interface allows users to upload a currency note image and receive a predicted denomination class with confidence scores, directly within the notebook environment.

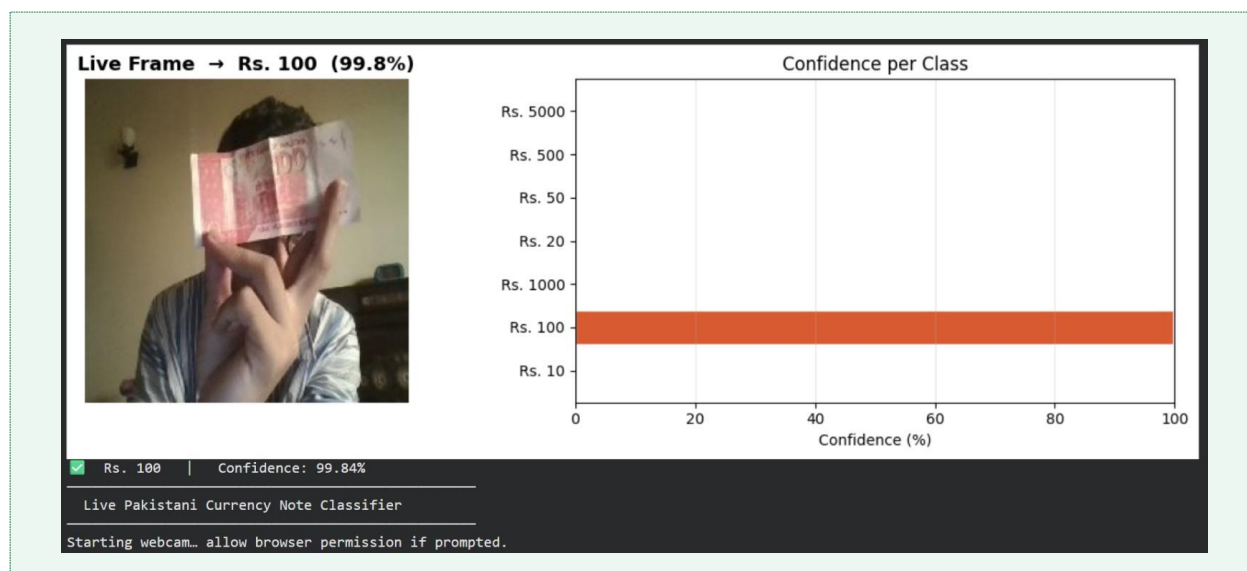
Tab / Feature	Description
Image Upload	Upload any currency note image from disk (JPG/PNG/JPEG). Preprocessed automatically (resize to 224×224, normalize).
Prediction Output	Displays predicted class (denomination + side), confidence score, and class probabilities for all 14 classes.
Reset Button	Clears the current input and output for a fresh prediction.

Table 9.1 — Interactive interface features

9.1: we also gave option of using live computer feed to predict note. Our webcam turns on and use discrete frames to predict the note.



Screenshot 20 — Interactive notebook interface showing upload tab and prediction result

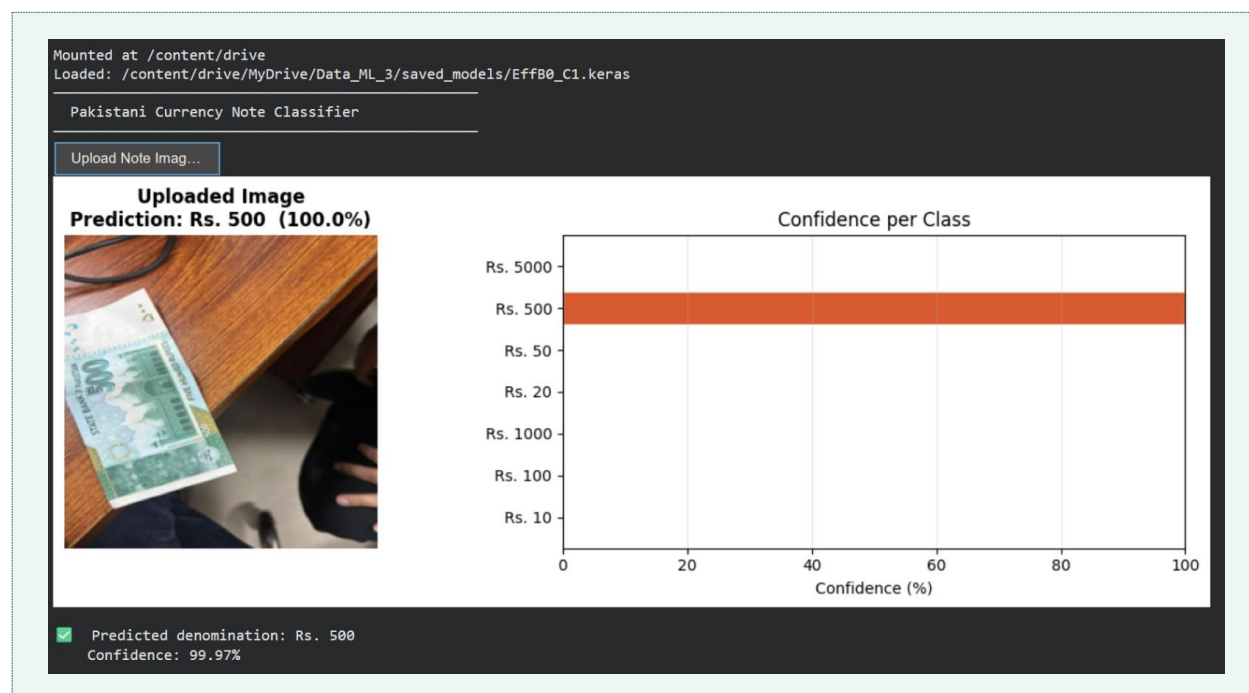


Screenshot 21 — Per-class confidence probability display in the interactive interface

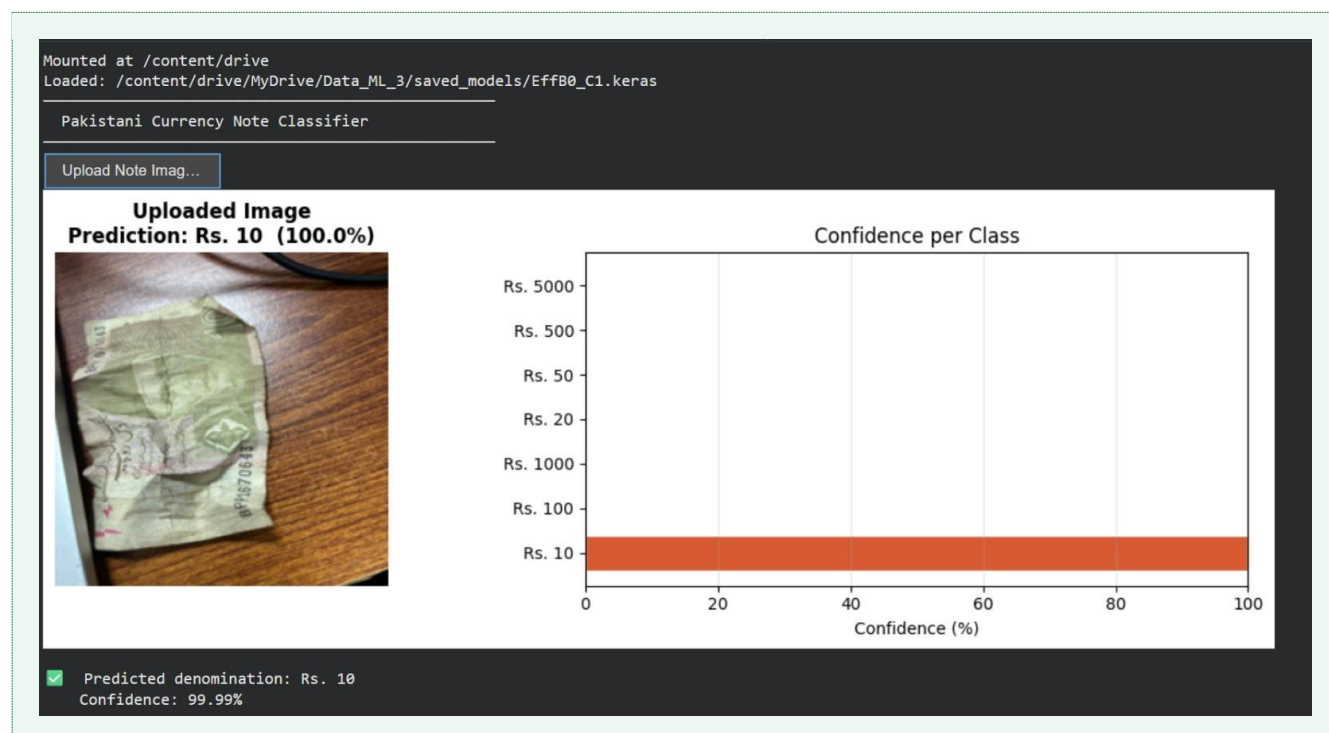
10. Test Case Runs

The following test cases demonstrate the deployed model's predictions on real currency note images using the interactive interface. Confidence scores and predicted classes are shown for each test case.

Test Case 1

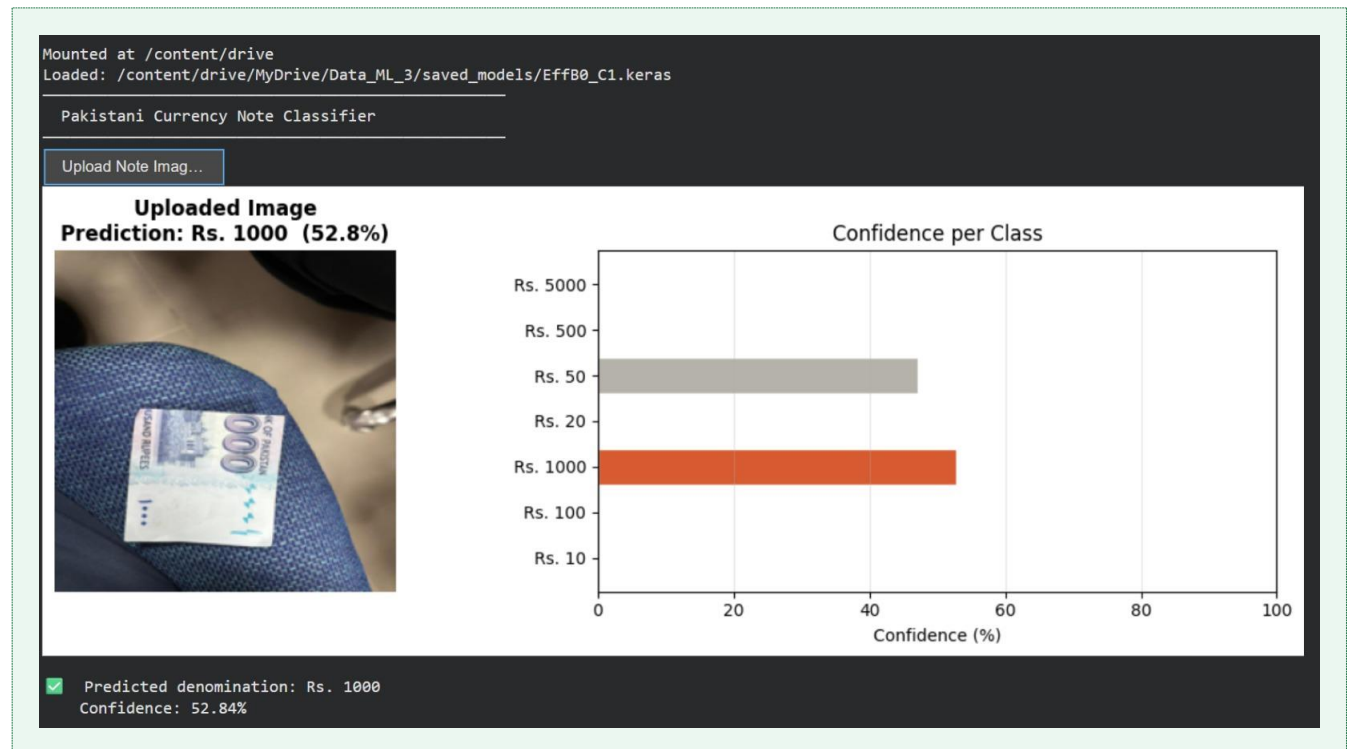


Test Case 2



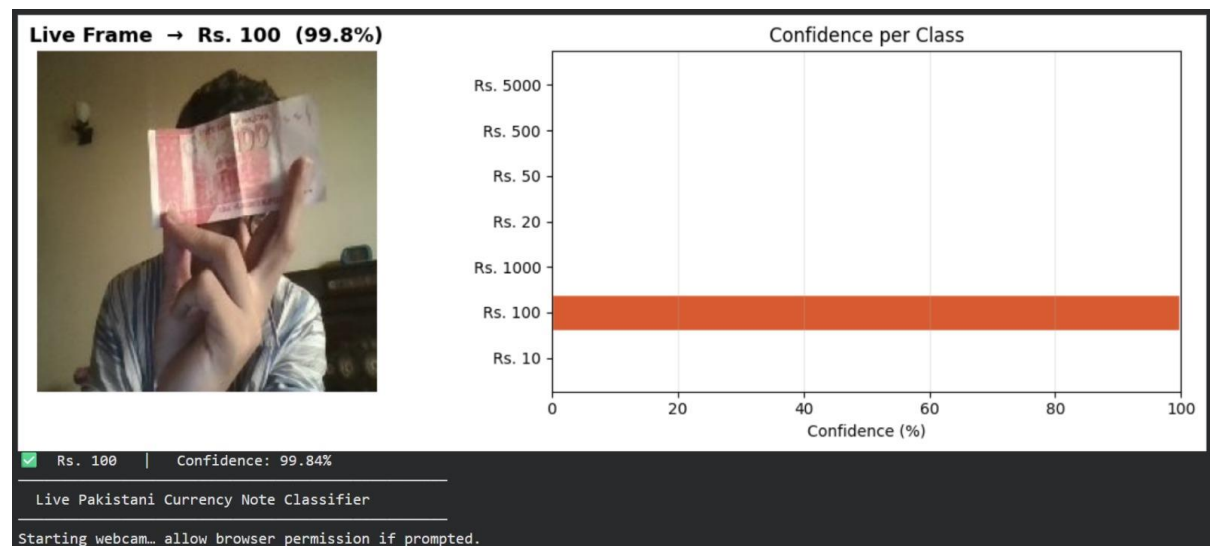
Screenshot 23 — Test Case 2: PKR 500 back note

Test Case 3

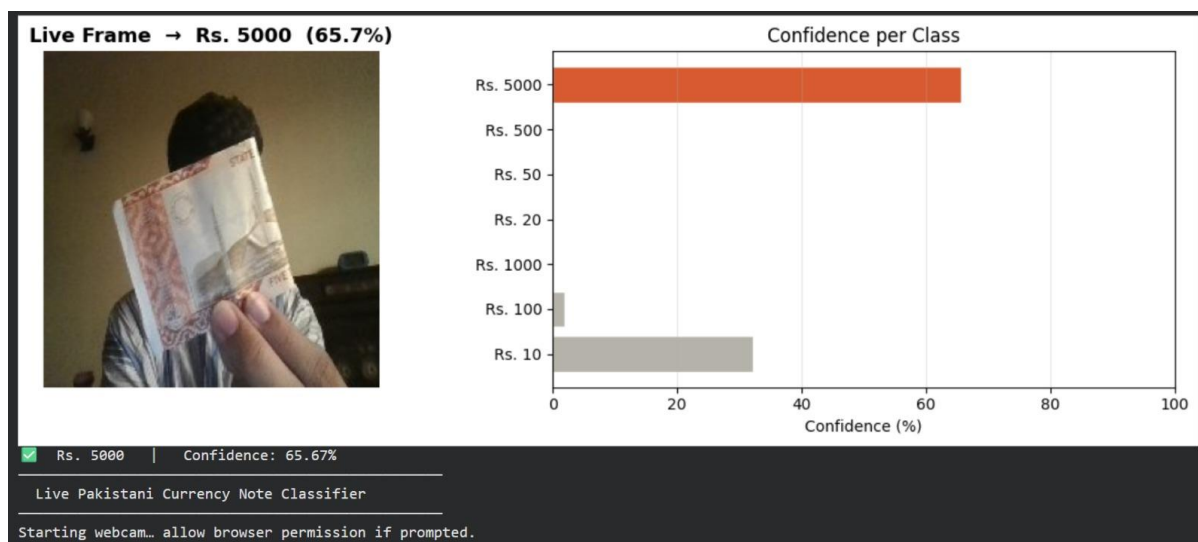


Screenshot 24 — Test Case 3: PKR 1000 front note

Test Case 4



Test Case 5



Test Case7



11. Unique & Notable Aspects

The following features make this implementation stand out beyond the minimum project requirements:

Feature	Description
Self-Collected Real-World Dataset	All 1,453 images were captured manually using smartphones under diverse real-world conditions — different lighting, angles, distances, and backgrounds. No synthetic or publicly available benchmark datasets were used.
7-Class Multi-Side Classification	The system classifies both the front and back of all 7 Pakistani denominations (PKR 10, 20, 50, 100, 500, 1000, 5000), going

Feature	Description
	beyond simple denomination recognition to provide a complete note identification system.
Comprehensive Preprocessing Pipeline	A 10-step preprocessing pipeline was implemented including format standardization, corrupted image removal, duplicate detection via perceptual hashing, spatial normalization, RGB standardization, one-hot encoding, and data augmentation.
Three Algorithmic Families	Nine models across Custom CNN, MobileNetV2, and EfficientnetB0 provide a thorough comparative study of from-scratch vs. transfer learning approaches on the same dataset.
Split Ratio Investigation	Both 70/15/15 and 80/10/10 splits were explored and compared quantitatively on the best-performing model.
Interactive ipywidgets Interface	A functional in-notebook interface allows real-time currency note classification with confidence scores, supporting all 14 class categories with a user-friendly upload mechanism.
Live computer vision field	Model takes in discrete frames from live feed and then predicts note using our selected model.
Comprehensive EDA	Class distribution charts, sample image grids, pixel intensity histograms, and image size distributions were all produced and analyzed before model training.

Table 11.1 — Unique and notable features of this implementation

12. Future Enhancements

The current system meets all project requirements and demonstrates strong performance. However, several enhancements could be applied to extend its capabilities for real-world deployment:

12.1 Model Improvements

- **EfficientNet Architecture:** Replace MobileNetV2 with EfficientNet-B0 or B3, which typically achieves higher accuracy with fewer parameters, potentially improving the accuracy-efficiency trade-off.
- **Ensemble Modeling:** Combine predictions from MobileNetV2 and EfficientnetB0 through averaging or stacking, which typically increases accuracy by 1–3% on classification tasks.
- **Attention Mechanisms:** Incorporate Squeeze-and-Excitation (SE) blocks or Convolutional Block Attention Module (CBAM) into the custom CNN to help the model focus on the most discriminative regions of the currency note.
- **Vision Transformer (ViT):** Experiment with ViT or DeiT architectures for potentially higher accuracy through global attention over the entire image, which could capture the holistic features of currency notes.

12.2 Dataset Expansion

- **Increase Dataset Size:** Collecting 300–500 images per class (vs. the current ~100–110) would significantly reduce overfitting in custom CNN models and improve overall generalization.
- **Include Worn/Damaged Notes:** Collecting images of damaged, torn, or worn currency notes would make the system robust to the realistic conditions in which currency recognition is most needed (e.g., ATMs, vending machines).
- **Include Counterfeit Variations:** Adding a 'counterfeit' class would transform the system into a genuine currency authentication tool with practical real-world utility.
- **Diverse Capture Conditions:** Capturing under more varied conditions such as direct sunlight, artificial fluorescent light, and low-light environments would further improve generalization.

12.3 Training Pipeline Improvements

- **K-Fold Cross-Validation:** Implementing 5-fold or 10-fold cross-validation would provide more statistically reliable performance estimates, particularly important given the dataset's moderate size.
- **Learning Rate Schedulers:** Applying cosine annealing or ReduceLROnPlateau across all models to dynamically adjust the learning rate and improve convergence.
- **Mixed Precision Training:** Using FP16 (half precision) training would reduce memory usage and increase training speed by 2–3x on compatible hardware.
- **Incremental Learning:** Implementing incremental learning capabilities to update the model with new currency denominations or new data without retraining from scratch.

12.4 Deployment & Interface Enhancements

- **Mobile Application:** Converting the best model to TensorFlow Lite (.tflite) and deploying it as an Android/iOS application would make it accessible to everyday users for quick currency identification.

- **Real-Time Camera Feed:** Implementing a continuous video stream input (rather than single image upload) would enable real-time currency recognition in production environments such as POS systems or ATMs.
- **Urdu Language Support:** Adding Urdu language labels and instructions in the interface would improve accessibility for Pakistani users who are less comfortable with English.
- **Grad-CAM Visualization:** Integrating Gradient-weighted Class Activation Mapping (Grad-CAM) would highlight which regions of the currency note the model uses for classification, improving interpretability and trust.
- **REST API Deployment:** Deploying the model as a REST API (using FastAPI or Flask) would allow integration with other applications, POS systems, or banking software.

13. Generative AI Use Declaration

Field	Details
AI Tools Used	ChatGPT (OpenAI), GitHub Copilot (Microsoft), Google Gemini
Purpose — Idea Generation	Exploring augmentation strategies, model architecture variants, and interface design ideas.
Purpose — Debugging Assistance	Resolving TensorFlow/Keras API usage issues, fixing data generator configurations, and debugging training loops.
Purpose — Code Formatting	Improving code readability, adding docstrings, and organizing notebook cells for clarity.
Purpose — Report Writing	Assistance with formal academic language, structure, and grammar in report sections.
Confirmation	All final design decisions, model implementations, training, evaluation, and analysis were performed by the student group. All submitted code and results are understood and defensible by all group members.

Table 13.1 — Generative AI Use Declaration